

NAME:

ID:

Apr 23, 2022

COMPREHENSIVE EXAMINATION

70 M, 3hrs

PART A (Closed Book)

25 M, 1hr

- PART A is in-built; please write your answers in the space provided
- Ensure you have both sides of PART A printed
- Assume all required header files and return statements are included for the C codes given.

I. State true or false for each statement, with justification. No mark shall be awarded without proper justification. [7]

i. The C library function `malloc( )` returns a pointer of type `int`.

False. It returns (void*) pointer.

ii. The C library function `sizeof( )` accepts only a single argument and returns an integer data type.

False. `sizeof( )` is an operator in C & not a function.

iii. `typedef` is a user-defined data type allowed to use in C programming.

False, `typedef` is not a data type.

iv. An example of correct declaration of static global variables is `extern static int a;`

False, `extern` and `static` don't go together (compilation error)

v. Using the `static` keyword before a function name restricts access to that function to the file where it is declared.

True.

vi. A pointer to a ten element character array can be declared as `char *p[10];`

False. `p` in the declaration is an array of pointers to `char`.

vii. The following is the correct way of assigning values to an array:

```
int arr[ ] = {0x64, 0xAB0, 0xA0A0, 0xFAFA, 0x100};
```

True. It initializes `arr` with 5 elements.

II. Answer the following questions briefly

[14]

a. Consider `pch`, `pshort` and `pdouble` declared as pointers to `char`, `short int` and `double`, respectively. Arrange the three variables in increasing order of size. Explain your answer.

Size of `pch` = size of `pshort` = size of `pdouble`. This is because all the three pointer variables ~~are~~^{store} addresses which is typically 8 bytes long.

b. For each of the following two questions, state whether the given two expressions are equivalent and/or valid. Assume the identifiers represent integer arrays.

(i) `&a[0]` and `a`

True. The name of the array `a` refers to the address of the zeroth element which is `&a[0]`.

(ii) `*amp(arr[1])` and `&*amp(arr[1])`

False `*amp(arr[1])` is same as `arr[1]` but `&*amp(arr[1])` is invalid since address of can't be used for value `*amp(arr[1])`.

c. What does the following piece of code accomplish?

```
int foo(int p, int q){
    if (q == 0) return 1;
    return p * foo(p, q-1);
}
```

It computes the value of p^q

d. Study each of the following pieces of code and state

the output. If there is an error, then state the type of error and the reason for the error;

```
i)
int *return_arr()
{
    int arr[] = {1, 4, 5};
    return arr;
}

int main()
{
    int *p = return_ptr();
    printf("%d ", *p);
}
```

Answer: Run time error.

cannot return array created locally inside the function (on stack) `return_arr()` as its lifetime is only till the end of the function scope.

```
ii)
int main() {
    int arr[] = {1,2,3,4,5};
    int *brr = arr;
    brr = arr;
    for(int i=0; i<5; i++) {
        brr[i]++;
    }
    for(int i=0; i<5; i++) {
        printf("arr[%d] = %d\t", i, arr[i]);
        printf("brr[%d] = %d\n", i, brr[i]);
    }
}
```

Answer: $arr[0] = 2$ $brr[0] = 2$
 $arr[1] = 3$ $brr[1] = 3$
 $arr[2] = 4$ $brr[2] = 4$
 $arr[3] = 5$ $brr[3] = 5$
 $arr[4] = 6$ $brr[4] = 6$

```
iii)
int main() {
    int arr[10] = {0}, n, i;
    scanf("%d", &n);
    n = n < 0 ? -n : n;
    while (n) {
        arr[n%10]++;
        n /= 10;
    }
    for (i = 0; i < 10; ++i)
        printf("%d : %d.\n", i, arr[i]);
}
```

Answer: It computes the frequency of occurrence of each of the 10 digits in an integer which is input by the user. It also prints out those frequencies at the end.

III. Write down the output of the following snippets of code. If you think there's some bug in the code, you may write it as well [4]

```
a)
int y=-1;
if (y==(-1))
    printf("Computer ");
else {y!=-1};
printf("Programming");
```

Output: Computer Programming

```
b)
static int num1;
char num2 = '5';
printf("%d %d\n", ++num1, -num2);
```

Output: 1 -6

```
c)
int i;
double d;
i = -5, -37, -77.5;
d = {0.319, -3.44, -1.111};
printf("%lf", i-d);
```

Output: -3.589

1. Given the following definitions for a linked list, write a function reverse () which reverses a given linked list. [8]

//Assume these are global definitions/declarations

```
struct list {
    int data;
    struct list *next;
};
typedef struct list node;
node *head; // assume that 'head' is
populated and points to the first node
```

node of the linked list

// function to reverse a linked list

```
void reverse()
{
    node* prev = NULL;
    node* current = _____;
    node* next = NULL;
    while (current != NULL) {
        _____
        _____
        _____
    }
    head = prev;
} // note: one blank is one line of code
```

Sample input: 1->2->3->4->5->NULL

Output: Linked list should be changed to,
5->4->3->2->1->NULL

2. The code given below is meant to generate a pattern as shown in the sample runs given below. Complete the code to generate the desired pattern. [8]

```
#define N 7 // Assume N <= 8
int main() {
    int n = N;
    int i, j; // Do NOT declare any new variables
    for( i=n ; i>=1 ; i-- ) {
        for( j=n ; j>=1 ; j-- ) {
            if( i==1 || i==n )
                _____
            else if ( j==1 || j==n )
                _____
            else if( j==n-i+1 || i==j )
                _____
            else
                _____
        }
        printf("\n");
    }
    return 0;
}
```

N=6	N=7
123421	1234321
53~45	68~46
4~3~4	5~3~5
4~3~4	4~2~4
53~45	5~3~5
123421	68~46
123421	1234321

Sample Runs

- ✓ 3. A rational number of the form p/q where p and q are integers (q not equal to 0) is to be stored as an entity to do useful operations like addition of two rational numbers and reducing a rational number to its lowest form. For example, $12/8$ after reducing to lowest form results in $3/2$. To achieve these tasks:

- ✓ a. Design a suitable datatype for storing a rational number
- ✓ b. Write two function prototypes for addition and reduction
- ✓ c. Implement the prototypes by writing their definition.

You may use the function `int GCD( int a, int b);` which returns the greatest common divisor of a and b . [9]

- ✓ 4. Answer the following questions: [4]

- ✓ a. What does the following piece of code accomplish?

```
int main(int argc, char *argv[]){
    for(int i=0; i< argc-1; i++){
        FILE *fp = fopen(argv[i+1], "r");
        char ch;
        while ((ch = fgetc(fp))!=EOF)
            fputc(ch, stdout);
        fclose(fp);
    }
}
```

- ✓ b. Write the most appropriate unix command (complete) that is equivalent of this piece of code.

- ✓ 5. Complete the program given below which prints all the combinations of two words in a dictionary which when combined together form a meaningful word in the dictionary. You may assume that all the words in the dictionary are of lower case. Here is a sample output the program should give: [16]

```
out + pour = outpour
side + kick = sidekick
under + stand = understand
```

```
#include <stdio.h>
#include <stdlib.h>
#include <string.h>
#define MAX 50
int isDictEntry(char*,char*[],int);
```

```
int main()
{
    char w1[MAX],w2[MAX];
    char combined[MAX];
    int i,j,N;
    scanf("%d",&N); // how many words
    char *words[N];
```

// Read words and store them using dynamic memory

```
for (i=0; i<N; ++i){
    scanf("%s",w1);
    words[i] = _____ (a) _____
    _____ (b) _____
}
```

/* Check and print if there are two words s_1 and s_2 such that s_1s_2 is also present in the dictionary. You may call the function `isDictEntry()` as required*/

```
_____ (c) _____
_____ (d) _____
_____ (e) _____
_____ (f) _____
_____ (g) _____
_____ (h) _____
```

*S append copy
check append
check
if-else*

//Max of 6 lines

```
for (i=0; i<N; ++i)
    _____ (i) _____ // Free unused memory
return 0;
} // End of main()
```

/* Returns 1 if word w is present in list of n words or 0 otherwise */

```
int isDictEntry(char*w, char*list[], int n)
{
    int i;
    _____ (j) _____
    _____ (k) _____
    _____ (l) _____
    _____ (m) _____
}
```